

ui.splitter 全面详细阐述

一、核心概述

`ui.splitter` 是 NiceGUI 框架中的一个布局组件，基于 Quasar 的 Splitter 组件开发，核心作用是将屏幕空间划分为可调整大小的多个区域，支持灵活且响应式的应用布局设计。通过该组件，用户可手动拖拽分隔条调整各区域尺寸，适配不同内容展示需求，同时提供了丰富的自定义配置项和扩展能力，适用于各类场景的界面布局。

二、核心特性与核心概念

（一）核心特性

- 双向分割支持**：默认支持垂直分割（左右布局），可通过参数配置为水平分割（上下布局）。
- 灵活的区域配置**：提供 `before`（前侧区域）、`after`（后侧区域）、`separator`（分隔条区域）三个可自定义插槽，可嵌入任意 NiceGUI 元素（文本、图片、组件等）。
- 精细的尺寸控制**：支持设置区域尺寸限制、初始尺寸，且可通过绑定机制实时同步尺寸值。
- 交互回调能力**：提供分割尺寸变化后的回调函数，支持响应式业务逻辑处理。
- 样式高度自定义**：支持通过 `classes`、`props`、`style` 等属性调整组件样式、布局细节。

（二）核心概念

- 插槽 (Slot)**：组件的可嵌入区域，`ui.splitter` 的三个核心插槽分工明确：
 - `before`：分割后的前侧区域（垂直分割时为左侧，水平分割时为上侧）；
 - `after`：分割后的后侧区域（垂直分割时为右侧，水平分割时为下侧）；
 - `separator`：分隔条区域，可自定义分隔条样式、图标等。
- 绑定 (Binding)**：通过组件的绑定方法（如 `bind_value`），可将分割尺寸与其他元素（如输入框）或变量实时关联，实现双向数据同步。

三、初始化参数

初始化 `ui.splitter` 时支持以下核心参数，用于定义组件的基础行为和布局：

参数名	说明	类型	默认值
horizontal	是否启用水平分割（默认垂直分割）	bool	False
limits	两个数字组成的列表，分别表示两个面板的最小和最大尺寸限制	tuple[float, float] / list[float, float]	-
value	第一个面板的初始尺寸（若启用 <code>reverse</code> 则为第二个面板的初始尺寸）	float	-
reverse	是否将 <code>value</code> 应用于第二个面板（而非第一个）	bool	False

参数名	说明	类型	默认值
on_change	回调函数，当用户释放分隔条（完成尺寸调整）时触发，参数为事件对象	Callable[[EventArguments], Any]	-

四、核心属性

`ui.splitter` 继承了 NiceGUI 基础元素的属性，以下为常用核心属性：

属性名	说明	类型	
classes	组件的 CSS 类（支持 Tailwind、Quasar 类），用于调整样式	Classes[Self]	
props	组件的 Quasar 特性（HTML 属性），用于扩展组件行为（如设置区域样式）	Props[Self]	
style	组件的内联 CSS 样式，用于精细调整布局	Style[Self]	
value	当前分割尺寸（与初始化 <code>value</code> 一致，支持动态修改）	BindableProperty	-
visible	组件是否可见（支持绑定）	BindableProperty	True
enabled	组件是否启用（禁用后无法调整分割尺寸）	BindableProperty	True
html_id	组件在 HTML DOM 中的唯一 ID（版本 2.16.0 新增）	str	-

五、常用方法

`ui.splitter` 提供了丰富的方法用于交互控制、数据绑定和样式调整，以下为核心常用方法：

（一）绑定相关方法

用于将组件属性与其他对象或元素关联，实现数据同步：

- `bind_value(target_object, target_name='value', forward=None, backward=None, strict=None) -> Self`
 - 双向绑定：将组件的 `value`（分割尺寸）与目标对象的指定属性关联，双方值变化时自动同步。
 - 示例：将分割尺寸与输入框绑定，输入框修改时分割尺寸同步调整，反之亦然。
- `bind_value_from(target_object, target_name='value', backward=None, strict=None) -> Self`
 - 单向绑定（从目标到组件）：仅同步目标对象属性的值到组件 `value`。
- `bind_value_to(target_object, target_name='value', forward=None, strict=None) -> Self`
 - 单向绑定（从组件到目标）：仅同步组件 `value` 到目标对象属性。
- `bind_visibility(target_object, target_name='visible', ...) -> Self`
 - 双向绑定组件的可见性与目标对象属性。

(二) 状态控制方法

用于动态调整组件的启用、禁用、可见性等状态：

1. `enable()` $\rightarrow$ `None`：启用组件（允许调整分割尺寸）。
2. `disable()` $\rightarrow$ `None`：禁用组件（禁止调整分割尺寸）。
3. `set_enabled(value: bool)` $\rightarrow$ `None`：设置组件启用状态（`True` 启用，`False` 禁用）。
4. `set_visibility(visible: bool)` $\rightarrow$ `None`：设置组件可见性（`True` 显示，`False` 隐藏）。
5. `set_value(value: Any)` $\rightarrow$ `None`：手动设置分割尺寸。

(三) 样式与布局方法

用于调整组件样式和布局：

1. `classes(add=None, remove=None, toggle=None, replace=None)` $\rightarrow$ `Self`
 - 动态添加、移除、切换或替换组件的 CSS 类。
2. `style(add=None, remove=None, replace=None)` $\rightarrow$ `Self`
 - 动态添加、移除或替换组件的内联 CSS 样式。
3. `tooltip(text: str)` $\rightarrow$ `Self`：为组件添加 tooltip（提示文本）。

(四) 其他常用方法

1. `clear()` $\rightarrow$ `None`：移除组件的所有子元素（清空 `before`、`after`、`separator` 插槽内容）。
2. `delete()` $\rightarrow$ `None`：删除组件及其所有子元素。
3. `update()` $\rightarrow$ `None`：强制更新组件在客户端的渲染状态。
4. `on_value_change(callback)` $\rightarrow$ `Self`：添加值变化回调（与 `on_change` 参数功能一致，更简洁的写法）。

六、使用示例

(一) 基础用法：简单垂直分割

```
from nicegui import ui

with ui.splitter() as splitter:
    # 前侧区域（左侧）
    with splitter.before:
        ui.label('左侧内容区域').classes('mr-2 p-4 bg-gray-100')
    # 后侧区域（右侧）
    with splitter.after:
        ui.label('右侧内容区域').classes('ml-2 p-4 bg-gray-200')

ui.run()
```

- 效果：页面分为左右两个可拖拽调整的区域，分别显示不同背景色的文本。

(二) 高级用法：自定义分隔条、尺寸绑定与回调

```
from nicegui import ui

# 创建分割器，设置初始尺寸、垂直分割、值变化回调
with ui.splitter(horizontal=False, reverse=False, value=60,
                 on_change=lambda e: ui.notify(f'分割尺寸: {e.value}')) as splitter:
    # 添加提示 tooltip
    ui.tooltip('可拖拽分隔条调整尺寸').classes('bg-green')

    # 前侧区域
    with splitter.before:
        ui.label('左侧区域').classes('mr-2 p-4')
    # 后侧区域
    with splitter.after:
        ui.label('右侧区域').classes('ml-2 p-4')
    # 自定义分隔条（添加灯泡图标）
    with splitter.separator:
        ui.icon('lightbulb').classes('text-green text-xl')

# 绑定分割尺寸到数字输入框，支持手动输入调整
ui.number('分割尺寸', format='%.1f').bind_value(splitter)

ui.run()
```

- 关键特性：
 - 自定义分隔条为绿色灯泡图标；
 - 初始左侧区域尺寸为 60；
 - 拖拽分隔条释放后，弹出提示显示当前尺寸；
 - 数字输入框与分割尺寸双向绑定，输入框修改时尺寸同步调整。

(三) 图片对比示例：左右图片分割展示

```
from nicegui import ui

# 设置分割器大小，隐藏图片溢出
with ui.splitter().classes('w-96 h-64') \
    .props('before-class=overflow-hidden after-class=overflow-hidden') as splitter:
    # 前侧区域：彩色图片
    with splitter.before:
        ui.image('https://cdn.quasar.dev/img/parallax1.jpg').classes('w-full h-full object-cover absolute-top-left')
    # 后侧区域：黑白图片
    with splitter.after:
        ui.image('https://cdn.quasar.dev/img/parallax1-bw.jpg').classes('w-full h-full object-cover absolute-top-right')

ui.run()
```

- 效果：实现图片对比功能，拖拽分隔条可查看同一图片的彩色与黑白版本，适用于设计对比、图像编辑等场景。

(四) 水平分割示例

```
from nicegui import ui

# 启用水平分割（上下布局），设置尺寸限制（最小20，最大80）
with ui.splitter(horizontal=True, limits=(20, 80), value=30) as splitter:
    with splitter.before:
        ui.label('上侧区域').classes('mb-2 p-4 bg-blue-100')
    with splitter.after:
        ui.label('下侧区域').classes('mt-2 p-4 bg-blue-200')

ui.run()
```

- 效果：页面分为上下两个区域，拖拽水平分隔条可调整高度，且上侧区域高度限制在 20-80 之间。

七、适用场景

1. **内容分栏布局**：如后台管理系统的侧边栏与主内容区、文档阅读的目录与正文区。
2. **数据对比展示**：如图片对比、表格数据对比、文本版本对比。
3. **灵活交互界面**：如可调整大小的日志面板、参数配置面板与预览面板。
4. **响应式设计**：结合尺寸限制和绑定功能，适配不同屏幕尺寸下的内容展示需求。

八、注意事项

1. **尺寸单位**：`value` 和 `limits` 的数值单位默认为像素（px），无需手动添加单位。
2. **溢出处理**：当区域内容超出尺寸时，需通过 `overflow-hidden` 等样式控制溢出行为（如图片示例）。
3. **绑定兼容性**：绑定目标对象需是可修改的属性（如实例属性、字典键值），严格模式（`strict=True`）下会校验属性是否存在。
4. **版本兼容性**：`html_id` 属性需 NiceGUI 2.16.0 及以上版本，`toggle` 类操作需 2.7.0 及以上版本。
5. **性能优化**：频繁调整尺寸时，可通过 `throttle` 参数限制回调触发频率（适用于 `on` 方法绑定事件时）。